Lecture 07: Auditing Program Binaries

August 26, 2024

Overview

- Software Security
- Vulnerabilities
  - Stack Buffer Overflows
  - Heap Buffer Overflows
  - String Filters
  - Integer Overflows
  - Type Conversion Errors
- Case Study

Security

- Having control of the flow of information in a system
- Only as secure as its weakest link
- Vulnerability may be in 3rd-party component
- We entrust our computer and data to every program installed
- Unintentional bugs often more difficult to find than malicious software
- Few, if any, programs are “totally secure”
- Cat and mouse between black hats and white hats

Vulnerability

- Usually an unexpected code path leading to unexpected, sometimes devestating consequences
- Programs must take data from the outside world, and be prepared for anything
- A simple exploit may just cause the program to crash
  - Often the first result of fuzzing, a technique aiding in vulnerability analysis
- Hackers must craft very specific data to take control of a vulnerable program

Stack Buffer Overflow

- (Sometimes shorted to "Stack Overflow")
- Paper by Aleph1
- Results from lack of bounds checking when writing to a buffer on the stack (local variable)

Demo: Classic stack buffer overlow on Linux

Stack Buffer Overflow

- (Sometimes shorted to "Stack Overflow")
- Paper by Aleph1
- Results from lack of bounds checking when writing to a buffer on the stack (local variable)

Demo: Classic stack buffer overlow on Linux

- Stack Canaries
  - Overwrite local pointers
    - Compilers rearrange locals before buffers
- Data Execution Prevent (DEP) / NX bit
  - Return to libc → Return Oriented Programming (ROP)
    - Address Space Layout Randomization (ASLR)

Heap Overflows

- Heaps tend to be managed by linked lists
- This means user-supplied data near pointers
- The freeing of a corrupted block can give write-what-where
- Trickier because a block is usually allocated to fit the data

String Filters

- Relying on null termination instead of hard length limits
  - `strcpy()`
  - `strcat()`
- Passing user-supplied data as `format`
  - `printf(user_data)`
- Attackers may need to craft shellcode to survive string transformations, e.g., to Unicode

Integer Overflows

- A numerical overflow (usually on a bounds check) leading to buffer overflow

```
  push    esi
  push    100                     ; size
  call    malloc                  ; malloc()
  mov     esi, eax
  add     esp, 4
  test    esi, esi
  je      short 0040104E
  mov     eax,dword ptr [esp+C]
  cmp     eax,100
  jg      short 0040104E
  push    eax                     ; maxlen
  mov     eax,dword ptr [esp+C]
  push    eax                     ; src
  push    esi                     ; dst
  call    strncpy                 ; strncpy()
  add     esp,0C
0040104E:
  mov     eax,esi
  pop     esi
  retn
```

Arithmetic Operations on User-Supplied Sizes

```
size_t total_size = data_size + header_size;
if (total_size < max_size) {
  char* block = malloc(total_size);
  write_header(block);
  memcpy(&block[header_size], data, data_size);
}
```

- Check the `data_size` directly
- Or limit number of bits for user supplied size

Type Conversion Errors

- More signed vs unsigned stuff
- `movsx` vs `movzx`
- The bit length limiting precaution can be completely undone by a simple mistaken declaration
  - `allocate_object(char* data, short data_length)`
  - `allocate_object(char* data, unsigned short data_length)`

Case Study

CVE-2013-2729

- Adobe Reader X BMP/RLE heap corruption
- `http://www.exploit-db.com/exploits/26703/`
- `http://blog.binamuse.com/2013/05/readerbmprle.html`